Academic Credential Verification DApp

1. Project Title & Description

****HireMe: Academic Credential Verification DApp****

HireMe is a decentralized application designed to combat fraud in academic certifications. It allows universities to issue verfiied academic credentials on the Ethereum Blockchain. Students can securely share their credentials with employers and employers can instantly verify them without manual university confirmations.

Features:

Issue academic credentials on-chain

Unique credential hash generation

Public verification system

Credential revocation capability

Credential restoration (unrevoke)

Event logging for transparency

2. Team Members

- Hariharan NKS - 9599319

- Kusha Latha Azmeera- 8884869 Frontend developer

- Om

-Yushen

Smart Contract Architecture:

The university deploys the contract and is responsible for issuing, revoking and restoring credentials. It contains name, university, degree, field, student wallet address, revocation status, whether the credential exists. The credentials are stored in a unique credential hash.

The architecture is built on three pillars: **Data Integrity**, **Identity Management**, and **Verifiable State**.

1. Data Structure & Storage

The contract uses a `struct` named `Credential` to store data. In Solidity, a struct is a custom type that allows you to group different variables under one name.

- **The Mapping:** `mapping(bytes32 => Credential) private credentials;`
    - This acts as a high-speed database. The `bytes32` key is the "digital fingerprint" (hash) of the certificate.
    - Using a mapping is more gas-efficient than an array because the contract can jump directly to a specific record without searching through a list.

2. The Hashing Mechanism (The "Fingerprint")

When `issueCredential` is called, it computes a Keccak-256 hash. This is a one-way cryptographic function.

- **Input-Sensitivity:** If you change a single letter in the student's name (e.g., "John" to "Jon"), the resulting hash changes entirely.
- **Collision Resistance:** By including `block.timestamp`, the contract ensures that if a student graduates twice with the exact same degree title, each certificate gets a unique ID.

3. State Management (The Lifecycle)

A credential in this contract exists in one of three logical states:

- **Non-Existent:** The hash has never been registered (`exists` is false).
- **Active:** The hash exists and `revoked` is false.
- **Revoked:** The hash exists, but `revoked` is true (the degree is no longer valid).

Detailed Function Analysis

`issueCredential` (Write Operation)

This is the "Minting" process.

- **Logic:** It packages the student's metadata, generates the hash, and saves it to storage.
- **Safety Check:** It uses `require(!credentials[credentialHash].exists)` to ensure an admin doesn't accidentally overwrite an existing record.
- **Event:** It triggers `CredentialIssued`. This is vital for "off-chain" systems (like a website) to "listen" for new graduates and update a public directory automatically.

`revokeCredential` & `unrevokeCredential` (Update Operations)

Unlike traditional databases where you might "delete" a row, blockchain data is immutable.

- Instead of deleting, we change a **boolean flag** (`revoked`).
- This creates a permanent audit trail. Even if a degree is revoked, the record that it *once existed* remains on the ledger, which is crucial for legal and academic transparency.

`verifyCredential` (Read Operation)

This is the most frequently used function by the public.

- **Efficiency:** It returns a simple `bool` (true/false).
- **Logic:** It checks two conditions: `c.exists && !c.revoked`. If the hash doesn't exist in the system, or if it has been revoked by the university, the verification fails.

Access Control & Security

The contract uses the **Ownable** pattern (simplified here as `onlyAdmin`).

- **Security Risk:** The `admin` address is a "Single Point of Failure." If the university's private key is stolen, an attacker could issue fake degrees or revoke real ones.
- **Improvement Path:** In a production environment, the `admin` would likely be a **Multi-Signature Wallet** (requiring 3 out of 5 deans to sign off) rather than a single wallet address.

```
AcademicCredentials Contract
│
├── State Variables
│     ├── admin (address)
│     └── credentials (mapping)
│
├── Structs
│     └── Credential { name, university, degree, field, student, revoked, exists }
│
├── Events
│     ├── CredentialIssued
│     ├── CredentialRevoked
│     └── CredentialUnrevoked
│
├── Modifiers
│     └── onlyAdmin
│
├── Core Functions
│     ├── issueCredential()
│     ├── getCredential()
│     ├── revokeCredential()
│     ├── unrevokeCredential()
│     └── verifyCredential()
│
└── Access Control
      └── Admin = contract deployer
```

3. System Architecture

Roles:

-**University/Admin:** The deploying address and authorized issuer of credential

-**Students:** Credential Receiver

```

- **Frontend Layer:** Built using React, Vite, and aesthetic TailwindCSS glassmorphism.

- **Web3 Provider Layer:** MetaMask facilitates user connections and signs transactions.

- **Blockchain Layer:** Local Hardhat/Ganache instances executing EVM bytecode.

## 4. Technologies Used

- **Smart Contract:** Solidity (v0.8.20), Hardhat Framework

- **Frontend Development:** React.js, Javascript

- **Web3 Integration:** Ethers.js, MetaMask Wallet

- **Testing:** Hardhat, Mocha, Chai

Solidity is used to write the smart contract that manages academic credentials

React.js builds the user interface for issuing and verifying credentials

ethers.js connects the frontend with the blockchain

MetaMask allows the users to interact with the contract using their wallet

……                    -styles the frontend interface
```

5. Prerequisites

- **Node.js:** v18.0 or newer

- **MetaMask Extension:** Installed in Chrome browser. Ethereum wallet used to connect the frontend to the blockchain and sign the transactions

- **Git** to clone and manage the project repository

- **Hardhat** Ethereum development environment used for compiling, testing and deploying smart contracts

6. User Guide

```
![alt text](<Screenshot 2026-03-13 at 2.09.35 PM.png>)
```

1. Getting Started with Ganache; Click the NEW WORKSPACE button to open the advanced configuration options.
2. **Configuring the Ganache Server:**

```
![alt text](<Screenshot 2026-03-13 at 2.10.06 PM.png>)
```

Set Hostname to 127.0.0.1(localhost) and take note of port number to input into the wallet, which is 7545.

Assign the Network ID as 1337 which matches with the network ID of [config.js]

Click "Start" button in the top right corner to boot the local blockchain.

3. Click the "Add wallet" button at the bottom of the screen to import an individual private key generated by the local Ganache instance.

```
![alt text](<Screenshot 2026-03-13 at 2.20.40 PM.png>)
```

4. **Connecting the Wallet to Ganache:**

```
![alt text](<Screenshot 2026-03-13 at 2.23.56 PM.png>)
```

Navigate to Network Settings: Open the wallet, access the network dropdown menu, and select the option to add a custom network manually.

Network name:Ganache GUI

New RPC URL:http://127.0.0.1:7545 (Ensure this matches with the Ganache port).

Chain ID:1337

Currency symbol: `ETH`

Click Save. The wallet will now connect to our local blockchain, allowing us to import our test accounts and interact with our dApp.

5.**Using Ganache Test Accounts**

```
![alt text](<Screenshot 2026-03-13 at 2.40.26 PM.png>)
```

Once the Ganache workspace is running, goto the main Accounts Dashboard which shows 10 test accounts, each pre-funded with 100 fake ETH to cover gas fees during smart contract deployment and testing.

To access the test accounts, click the Key Icon on the far right of any account row in Ganache. Copy the revealed private key and paste it in the “Import Account” after opening the wallet. This will add that specific test account and its 100 ETH balance to our wallet.

6. **Deploying the Smart Contract:**

```
![alt text](<Screenshot 2026-03-13 at 2.41.17 PM.png>)
```

Once the Ganache workspace is running and the wallet is configured, we are ready to deploy the decentralized application (dApp) to our local blockchain. We use Hardhat to manage the compilation and deployment process.

- Open the terminal and navigate to the root directory of the project (e.g., `ACADEMIC-CREDENTIALS-DAPP`).
- Run the following command to deploy the contract to our local Ganache environment: ```bash

  npx hardhat run scripts/deploy.js --network ganache

7. **Updating the Frontend Configuration:**

```
![alt text](<Screenshot 2026-03-13 at 2.41.58 PM.png>)
```

After successfully deploying the smart contract, let's update the frontend application so it knows exactly where to find and interact with the contract on the local blockchain.

In the project directory, open the file designated for contract addresses and paste the contract address generated from the terminal deployment into the `"address"` field and ensure the `"network"` field is correctly set to `"ganache"`.

8. **Verifying Contract Deployment:**

```
![alt text](<Screenshot 2026-03-13 at 2.42.44 PM.png>)
```

After running the deployment script, it is good practice to verify that the transaction was successfully processed by your local blockchain.

Open the Ganache GUI and view the main ACCOUNTS dashboard.

Check the Block Height should have incremented from `0` to `1`indicating a new block was mined to include our contract.

Verify Gas Consumption from the TX COUNT should show at least `1` and the BALANCE should be slightly less than `100.00 ETH` (e.g., `99.85 ETH`). This reduction confirms that the network correctly charged the account for the gas fees required to deploy the smart contract.

9. **Inspecting Block and Transaction Details**

```
![alt text](<Screenshot 2026-03-13 at 2.42.56 PM.png>)
```

Navigate to the Blocks Tab and click on the most recently mined block to look at the transaction details within the block where we see a red CONTRACT CREATION badge.

10. **Exporting a Private Key for Wallet Import:**

```
![alt text](<Screenshot 2026-03-13 at 2.43.21 PM.png>)
```

To interact with our locally deployed smart contracts through a frontend interface, we will need to import at least one Ganache test account into our MetaMask wallet.

On the Ganache ACCOUNTS dashboard, locate the account we wish to use (typically Index 0, as it holds the remaining ETH from your deployment) and click the Key Icon on the far right of that row. Copy the Key and paste the copied private key into the designated fieldafter opening the wallet and selecting “Import Account” and click Import.

We should now see the imported account in our wallet, complete with its Ganache ETH balance, ready to sign transactions on our local dApp!

11. Verifying the Wallet Connection:

```
![alt text](<Screenshot 2026-03-13 at 2.45.28 PM.png>)
```

Open the Wallet and check the ETH balance under the “Tokens” tab and we should see a balance of 99.85 ETH reflecting the gas used during contract deployment.

We are now fully set up!The local blockchain is running, our smart contract is deployed, and our wallet is funded and connected. We can now launch the frontend application to interact with our dApp.

12. **Generating the Frontend Interface:**

```
![alt text](<Screenshot 2026-03-13 at 2.51.04 PM.png>)
```

Execute the following command :

```bash
```

node dapp-generator.cjs artifacts/contracts/AcademicVerification.sol/AcademicCredentials.json --address 0x44eab64A1296bf90831dCA7C1F85620b55d06c9C --out ./simple-frontend

(<Screenshot 2026-03-13 at 2.52.00 PM.png>)

13. Launching the Frontend Application:

```
![alt text](<Screenshot 2026-03-13 at 2.52.00 PM.png>)
```

From within the `simple-frontend` directory, execute the following command to start a lightweight web server on port 3000:

```bash

npx http-server -p 3000
```

14. **Importing Additional Test Users**

```
![alt text](<Screenshot 2026-03-13 at 2.52.59 PM.png>)
```

Return to the Ganache ACCOUNTS dashboard. Choose a different account from the list (e.g., the account at Index 1) and click its corresponding Key Icon. Copy the Private Key and paste the copied private key into the designated field and click Import.

(<Screenshot 2026-03-13 at 2.54.02 PM.png>)

15. **Connecting and Interacting with the DApp**

```
![alt text](<Screenshot 2026-03-13 at 2.54.02 PM.png>)
```

With the local server running, we can now connect our wallet to the frontend interface to begin issuing and verifying academic credentials. Navigate to `http://localhost:3000` in the web browser.

Click “Connect MetaMask” at the top of the interface.The MetaMask wallet extension will pop up, asking us to select an account and authorize the connection to this site.

Select the imported Ganache test account and approve the connection. The status indicator on the DApp will turn green and display the active wallet address.

To interact with the contract, use the `getCredential` or `verifyCredential` panels to query existing data. Use the `issueCredential` or `revokeCredential` panels to add or modify data on the blockchain. Submitting these forms will automatically trigger the wallet to pop up, asking to review and sign the transaction to pay the required gas fee.

16. Authorizing the Wallet Connection:

```
![alt text](<Screenshot 2026-03-13 at 2.54.14 PM.png>)
```

Before we can interact with the smart contract, we must authorize the connection between Metamask wallet and the frontend application. On the dApp interface, click the blue “Connect MetaMask”button.

After verifying that the requesting origin is our local server (`localhost`), select the university account and click the “Connect” button at the bottom of the MetaMask panel.

Once connected, the dApp interface will update. The red "Not connected" indicator will turn green, and our active public address will be displayed, indicating we are ready to execute contract functions.

17. **Verifying the DApp Connection:**

```
![alt text](<Screenshot 2026-03-13 at 2.54.26 PM.png>)
```

After approving the MetaMask prompt, look at the top-left status panel to be green, and it must display the public address of our imported Ganache test account.

Verify that the Chain ID is`1337`and the main connection button should now read “Connected”

(<Screenshot 2026-03-13 at 2.55.30 PM.png>)

18. Issuing a Credential (Writing to the Blockchain)

```
![alt text](<Screenshot 2026-03-13 at 2.55.30 PM.png>)
```

Locate the `issueCredential` panel and enter these details:

* **_student:** The public address of the receiving student (e.g., `0x5456Ac...`).

* **_name:** The student's name.

* **_university:** The issuing institution (e.g., `UOWD`).

* **_degree & _field:** The specific academic qualifications.

Click the **Call issueCredential()** button. The dApp will display a "Waiting for MetaMask confirmation..." status. The MetaMask extension will open a "Transaction request" panel, verify the details and click Confirm in MetaMask.

Once confirmed, the wallet will cryptographically sign the data and send it to Ganache. After a brief moment, the transaction will be mined into a new block, and the student's credential will be permanently recorded on local blockchain!

19. Verifying Transaction Success and Events:

```
![alt text](<Screenshot 2026-03-13 at 2.55.43 PM.png>)
```

Check the UI for the Transaction Hash. We should see a green **Confirmed** alert displaying a long hexadecimal string. This is our Transaction Hash (TxHash), a permanent receipt of our action on the blockchain.

Scroll down to the “Contract Events” section of the dApp and we should see a new log entry (e.g., `CredentialIssued` at block 2), confirming the contract's logic executed as intended.

(<Screenshot 2026-03-13 at 2.55.56 PM.png>)

20. Reading Contract Events and Extracting Hashes

```
![alt text](<Screenshot 2026-03-13 at 2.55.56 PM.png>)
```

Locate the Events Panel and scroll down to the **Contract Events** section at the bottom of the dApp interface. Review the log of the most recent `CredentialIssued` event. The panel will display a JSON object containing all the details that were just written to the blockchain (e.g., the student's address, name, university, and degree).

Find the `"credentialHash"` key within the JSON output. Copy and save its corresponding hexadecimal string (e.g., `0xe1bbcb...`).

(<Screenshot 2026-03-13 at 2.56.25 PM.png>)

21. Reading and Verifying Credentials

```
![alt text](<Screenshot 2026-03-13 at 2.56.25 PM.png>)
```

Once a credential is permanently written to the blockchain, anyone with the unique hash can verify its authenticity. Locate the `getCredential` panel under the **READ FUNCTIONS** section and paste the copied hash into the `_credentialHash` input field.

Click the “Call getCredential()” button. The interface will display the complete student record (name, university, degree, field) and confirm its current status (`exists: true`, `revoked: false`).

For a simple verification, paste the same hash into the `verifyCredential` panel.

Click **Call verifyCredential()**. It will return `true` if the credential is valid and active, or `false` if it does not exist or has been revoked by the issuer.

(<Screenshot 2026-03-13 at 2.56.58 PM.png>)

22. Revoking an Existing Credential

```
![alt text](<Screenshot 2026-03-13 at 2.56.58 PM.png>)
```

If a credential was issued in error or needs to be invalidated, the issuing authority can permanently flag it as revoked on the blockchain. Because this changes the contract's state, it requires a wallet signature and a gas fee.

Find the `revokeCredential` form under the **WRITE FUNCTIONS (REQUIRES METAMASK)** section of the dApp. Paste the specific `credentialHash` of the record we wish to invalidate into the input field.

Click the **Call revokeCredential()** button. The interface will temporarily pause and await wallet confirmation. The MetaMask wallet will pop up with a Transaction Request. Review the estimated gas fee and click **Confirm**.

Once the transaction is mined, the credential is permanently flagged. If anyone attempts to verify this specific hash using the `getCredential` read function, the `revoked` status will now return `true`, and the quick `verifyCredential` check will return `false`.

(<Screenshot 2026-03-13 at 2.57.11 PM.png>)

23. Verifying Credential Revocation:

```
![alt text](<Screenshot 2026-03-13 at 2.57.11 PM.png>)
```

To verify the state of blockchain after confirming the revocation transaction in the wallet, we must verify that the blockchain state was successfully updated.

Check the `revokeCredential` button. A green **Confirmed** alert should appear, displaying the transaction's unique hash. This is our cryptographic proof that the revocation order was processed.

Open the MetaMask extension and check the **Activity** tab. We should see a new "Contract interaction" marked as **Confirmed**, indicating the gas fee was paid and the network accepted the state change.

Paste the revoked `credentialHash` into the `getCredential` input and call it. The output parameter for `revoked` should now display `true`.

Paste the hash into the `verifyCredential` input and call it. The output should now return `false`, actively preventing this degree from being authenticated by third parties.

24. Confirming Revocation Status

```
![alt text](<Screenshot 2026-03-13 at 2.57.30 PM.png>)
```

Once a revocation transaction is confirmed on the network, it is best practice to verify that the blockchain state reflects this update. You can use the **READ FUNCTIONS** to check the new status of the credential. Navigate to the `getCredential` panel and input the revoked `credentialHash`.

Click **Call getCredential(). Review the output data. We will see that the original data (student name, university, degree) remains intact, but the `revoked` flag is now set to `true`.

Navigate to the `verifyCredential` panel and input the same hash. Click **Call verifyCredential()**. The function will now return `false`. This ensures that any automated systems or third parties checking this specific credential hash will correctly recognize it as invalid.

(<Screenshot 2026-03-13 at 2.57.48 PM.png>)

25.Reinstating a Revoked Credential (Unrevoke):

```
![alt text](<Screenshot 2026-03-13 at 2.57.48 PM.png>)
```

If a credential was revoked by mistake, or if a student has resolved the issue that led to the revocation, the issuing authority can reinstate the credential's validity on the blockchain.

Locate the Unrevoke Panel to find the `unrevokeCredential` form within the **WRITE FUNCTIONS** section of the dApp.Paste the exact `credentialHash` of the previously revoked record into the input field and click the **Call unrevokeCredential()** button.

Click **Confirm**.

Once the transaction is confirmed, scroll down to the **Contract Events** panel to verify that a `CredentialUnrevoked` event was emitted. You can also return to the **READ FUNCTIONS** to test the hash again; `verifyCredential` will now return `true`, and the `getCredential` output will show the `revoked` flag has returned to `false`.

26. Confirming Reinstatement Status

```
![alt text](<Screenshot 2026-03-13 at 2.58.09 PM.png>)
```

Once an unrevoke transaction is confirmed on the network, you should verify that the blockchain state reflects this restoration. Navigate to the `getCredential` panel and input the reinstated `credentialHash`. Click **Call getCredential()**.

Review the output data. You will see that the `revoked` flag is now set back to `false`.

Navigate to the `verifyCredential` panel and input the same hash.

Click **Call verifyCredential()**.

The function will now return `true`. This ensures that any automated systems or third parties checking this specific credential hash will correctly recognize it as an active, valid degree.

(<Screenshot 2026-03-13 at 2.58.59 PM.png>)

27. Auditing Contract Calls in Ganache

```
![alt text](<Screenshot 2026-03-13 at 2.58.59 PM.png>)
```

Whenever you execute a issuing or revoking a credential function from the frontend dApp, you can audit the exact on-chain execution details using the Ganache GUI.

After executing a function in the dApp, copy the resulting TxHash from the green success notification. Open Ganache and click on the **BLOCKS** or **TRANSACTIONS** tab.

Verify that the **TO CONTRACT ADDRESS** matches our deployed `AcademicCredentials` contract address. We can also review the **GAS USED** here to monitor the computational efficiency of our specific smart contract functions.

```
![alt text](image.png)

![alt text](image-1.png)

![alt text](image-2.png)

![alt text](<Screenshot 2026-03-13 at 3.03.33 PM.png>)



![alt text](<Screenshot 2026-03-13 at 2.59.09 PM.png>)

![alt text](<Screenshot 2026-03-13 at 2.59.17 PM.png>)

![alt text](<Screenshot 2026-03-13 at 2.59.30 PM.png>)

![alt text](<Screenshot 2026-03-13 at 3.02.23 PM.png>)

![alt text](<Screenshot 2026-03-13 at 3.00.56 PM.png>)

![alt text](<Screenshot 2026-03-13 at 3.01.11 PM.png>)

![alt text](<Screenshot 2026-03-13 at 3.01.29 PM.png>)

![alt text](<Screenshot 2026-03-13 at 3.02.40 PM.png>)
```

```
![alt text](<Screenshot 2026-03-13 at 3.03.08 PM.png>)

![alt text](<Screenshot 2026-03-13 at 3.03.53 PM.png>)

![alt text](<Screenshot 2026-03-13 at 3.04.23 PM.png>)

![alt text](<Screenshot 2026-03-13 at 3.04.34 PM.png>)

![alt text](<Screenshot 2026-03-13 at 3.04.51 PM.png>)
```

1. Clone the repository

git clone https://github.com/username/credential-dapp.git

cd credential-dapp

2. Install all required packages

npm install

3. Compile smart contracts

npx hardhat compile

4. Start Local Blockchain

npx hardhat node

Or

![ganche window](<Screenshot 2026-03-13 at 2.09.35 PM.png>)

5. Deploy Smart Contract

npx hardhat run scripts/deploy.js --network localhost

6. Run the frontend

npm start

7. Open the application

http://localhost:3000

8. Connect to your MetaMask wallet to interact with the smart contract

7. Smart Contract functions

constructor()- initializes the contract and controls credential issuance and revocation

issuecredential() - Admin only

```
function issueCredential(
    address _student,
    string memory _name,
    string memory _university,
    string memory _degree,
    string memory _field
) public onlyAdmin returns (bytes32 credentialHash)
```

Stores the credential data,generates unique hash

getcredential()

```
function getCredential(bytes32 _credentialHash) public view returns
```

Retrieves full credential information

revokeCredential() - Admin only

```
function revokeCredential(bytes32 _credentialHash) public onlyAdmin
```

Revokes a previously issued credential

unrevokeCredential() - Admin only

```
function unrevokeCredential(bytes32 _credentialHash) public onlyAdmin
```

Ensures the credential exists, revoked and restores it to active status

verifyCredential()

```
function verifyCredential(bytes32 _credentialHash) public view returns (bool)
```

Verifies if a credential is valid

Events

CredentialIssued

When a credential is created

```
event CredentialIssued(
    bytes32 credentialHash,
    address student,
    string name,
    string university,
    string degree,
    string field
);
```

CredentialRevoked

When a credential is revoked

```
event CredentialRevoked(bytes32 credentialHash);
```

CredentialUnrevoked

When a credential is restored

```
event CredentialUnrevoked(bytes32 credentialHash);
```

Sample workflow

1. University issues credential

issueCredential(studentAddress, "Kusha", "UOWD", "BE", "Electronics")

2. Contract returns acredential hash

0x8b21a4e5...

3. Employer verifies credential

verifyCredential(credentialHash)

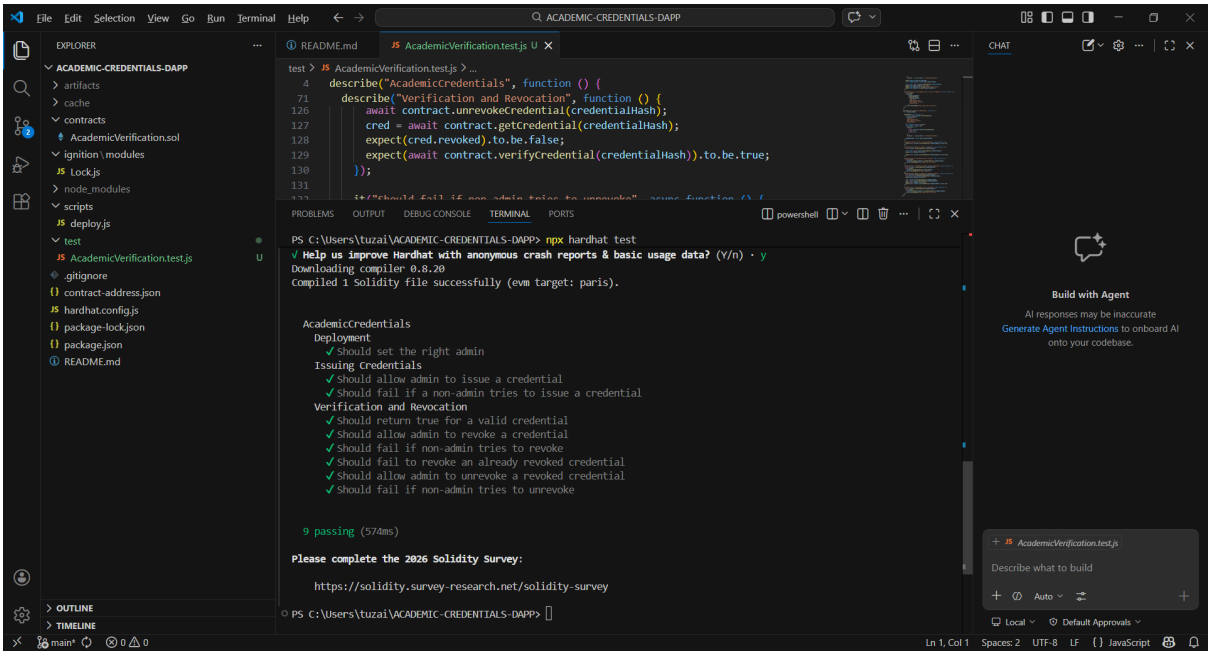
Returns: true

4. If credential is revoked

verifyCredential(credentialHash)

Returns: false

8. 🧪 Testing Instructions



1. Prerequisites

Before running the test suite, ensure your local development environment is ready:

* Install all project dependencies by running `npm install` in the root directory.

* The test suite will automatically spin up a local in-memory Hardhat network, so no external Ganache instance is required for this step.

2. Execution Command

To execute the automated security and logic tests, run the following command in your terminal:

bash

npx hardhat test

3. Test Coverage & Security Assertions

The test suite consists of **9 passing assertions** that thoroughly verify both "happy paths" (expected behavior) and adversarial edge cases (revert testing):

* **Deployment:** Verifies that the contract deploys successfully and correctly assigns the deployer's address as the singular authorized `Admin`.

* **Issuing Credentials:** * Confirms the Admin can successfully issue a credential and store the cryptographic hash.

  * **Security Check:** Asserts that the transaction *fails and reverts* if a non-admin attempts to issue a credential.

* **Verification and Revocation Lifecycle:**

  * Validates that the `verifyCredential` function returns `true` for newly issued hashes.

  * Confirms the Admin can `revoke` a credential and subsequently `unrevoke` it, updating the state accurately.

  * **Security Check:** Asserts that the EVM strictly rejects non-admins attempting to execute `revoke` or `unrevoke` actions.

  * **Edge Case:** Ensures the system cleanly handles logical errors, such as failing gracefully if an Admin attempts to revoke an *already revoked* credential.

9.⚠️ **Known Issues/Limitations**

While this MVP successfully demonstrates the core lifecycle of credential issuance, verification, and revocation on a local blockchain, there are several architectural limitations inherent in the current design. Acknowledging these trade-offs is crucial for understanding the system's boundaries before considering a production-level deployment.

1. Prohibitive On-Chain Storage Costs (Scalability)

* **The Issue:** The smart contract currently stores raw string data (e.g., Student Name, University, Degree Title) directly within the contract's state on the EVM (Ethereum Virtual Machine).

* **The Impact:** As observed in the Ganache transaction logs, state-changing operations like `issueCredential` consume significantly high Gas (~151,647 Gas) compared to simple boolean updates like `revokeCredential` (~31,126 Gas). While cost-free on a local testnet, deploying this heavy data structure to the Ethereum Mainnet would result in exorbitant, unscalable costs for an institution issuing thousands of degrees annually.

2. Public Ledger Exposure (Data Privacy & Compliance)

* **The Issue:** Blockchains are inherently public ledgers. Currently, the system does not utilize off-chain encryption or Zero-Knowledge (ZK) proofs to mask the stored academic data.

* **The Impact:** Querying a valid `credentialHash` directly returns plaintext Personally Identifiable Information (PII) tied to a specific public wallet address. This poses severe data privacy risks (e.g., non-compliance with GDPR) and creates a scenario where a student's real-world identity could be correlated with their entire on-chain financial history.

3. Hardcoded Environment Constraints (UX & Adoption Friction)

* **The Issue:** The frontend architecture (`config.js`) currently statically binds the application to the `localhost` environment and a specific `EXPECTED_CHAIN_ID` (1337).

* **The Impact:** This design requires users to possess the technical knowledge to manually configure a Custom RPC network in their MetaMask wallet to interact with the DApp. In a production environment, this creates massive friction for non-technical users (students and HR verifiers), significantly hindering mass adoption and seamless user experience.

4. Centralized Single Point of Failure (Security)

* **The Issue:** The system's Access Control logic relies on a singular "University Admin" wallet (e.g., an `onlyOwner` or single `issuer` modifier) to execute all critical write functions.

* **The Impact:** Although the application utilizes decentralized infrastructure, its governance is highly centralized. This creates a severe Single Point of Failure (SPOF). If the university's single private key is lost, the institution loses all ability to issue or manage credentials. If compromised, malicious actors could forge or arbitrarily revoke legitimate degrees.

10. 🚀 Future Improvements

To evolve this DApp from a local proof-of-concept into a production-ready enterprise solution, the following upgrades are planned:

* **Off-Chain Metadata Storage (IPFS):** Transition from storing heavy string data (names, universities) directly on-chain to storing metadata on IPFS. The smart contract will only store the IPFS CID, drastically reducing gas costs for the institution.

* **Multi-Signature Admin Control:** Implement a Gnosis Safe (Multi-sig) contract for the University Admin role, requiring multiple department heads to sign off on a revocation to prevent unilateral errors or malicious actions.

* **Soulbound Tokens (SBTs):** Upgrade the credential architecture to utilize ERC-5192 (Soulbound Tokens). This would represent the degree as a non-transferable NFT directly in the student's wallet, allowing for native integration with Web3 professional networks.

* **Batch Issuance:** Develop a frontend script that allows the university registrar to upload a CSV file and issue hundreds of credentials in a single, gas-optimized transaction.